Software Architecture and Design Specification

(SAD)

Online Internship & Placement Preparation Portal

TEAM NO: 6

Project	Online Internship & Placement Preparation Portal
Version	1.0
Authors	PES2UG24CS186, PES2UG24CS190, PES2UG24CS208, PES2UG24CS214
Institution	PES University, Bengaluru — B.Tech CSE
Technology	MERN Stack (MongoDB, Express.js, React.js, Node.js) with Python for analytics
Date	25-09-2026
Status	Draft for Review

Revision History

Version	Date	Author	Change Summary	Approval
1.0	25-09-2026	PES2UG24CS186, PES2UG24CS190, PES2UG24CS208, PES2UG24CS214	Initial SAD covering architecture, component design, sequence diagrams, API design, and security architecture	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Prof. Sheela Devi		
Faculty Guide			

Table of Contents

1. Introduction

2. Document Overview

3. Architecture

4. Design

5. Appendices

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Online Internship & Placement Preparation Portal (IPP). It elaborates the IPP Software Requirements Specification (SRS) v1.0 into architectural and detailed-design decisions: component structure, chosen architecture pattern, technology stack, sequence flows, API contracts, security architecture, and error-handling/monitoring strategy.

1.2 Scope

Covers the architecture and design of all IPP services: the React.js client, the Node.js/Express REST API (authentication, listings, applications, aptitude test, and admin modules), the Python/Flask analytics and scoring microservice, MongoDB data storage, resume file storage, and the third-party email/SMS integration. Payment processing, third-party ATS integration, and post-selection payroll/HR onboarding are out of scope, consistent with the SRS.

1.3 Audience

Developers, QA engineers, security reviewers, the course instructor/evaluators, and future maintainers of the IPP codebase.

1.4 Definitions

IPP, MERN, JWT, RBAC, OTP, API, TLS, WCAG, STRIDE, ADR, RTM, CSV. See the Glossary (Section 5.1) for full expansions.

2. Document Overview

2.1 How to Use This Document

Section 3 (Architecture) describes system-level structure: goals and constraints, stakeholder concerns, the component diagram, the chosen architecture pattern and rationale, the technology stack, risks, requirement traceability, and the security architecture (STRIDE threat model). Section 4 (Design) descends to detailed design: UML sequence diagrams for key flows, REST API contracts, error-handling/logging/monitoring strategy, UX design notes, and open issues.

2.2 Related Documents

- IPP Software Requirements Specification (SRS) v1.0
- IPP Software Test Plan (STP) v1.0
- IPP Requirements Traceability Matrix (RTM) — SRS Section 8

3. Architecture

3.1 Goals & Constraints

Goals

- Secure, role-based access for Student, Recruiter, and Placement Officer/Admin roles
- 90% monthly uptime (IPP-NF-002) and ≤8s response time for 90% of requests (IPP-NF-001)
- WCAG 2.1 AA accessibility conformance (IPP-NF-004)
- Horizontal scalability of the API layer to absorb a 2x load increase (IPP-NF-005)

Constraints

- Must be built on the MERN stack, with Python used only for the analytics/scoring microservice
- Resume uploads limited to PDF, maximum 5MB
- Must comply with institutional data-privacy norms for student academic and personal data
- Academic project timeline and four-person team limit the scope of infrastructure complexity

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Students	Usability, data privacy, fast and reliable access to listings and test results
Recruiters / Companies	Ease of posting listings and reviewing applications
Placement Officers / Admin	Oversight, accurate institution-wide analytics, regulatory/institutional compliance
Developers (project team)	Modularity, maintainability (IPP-NF-006), clear API contracts
Course Coordinator / Evaluators	Traceability from requirements to architecture, design rigor, security awareness

3.3 Component (UML) Diagram

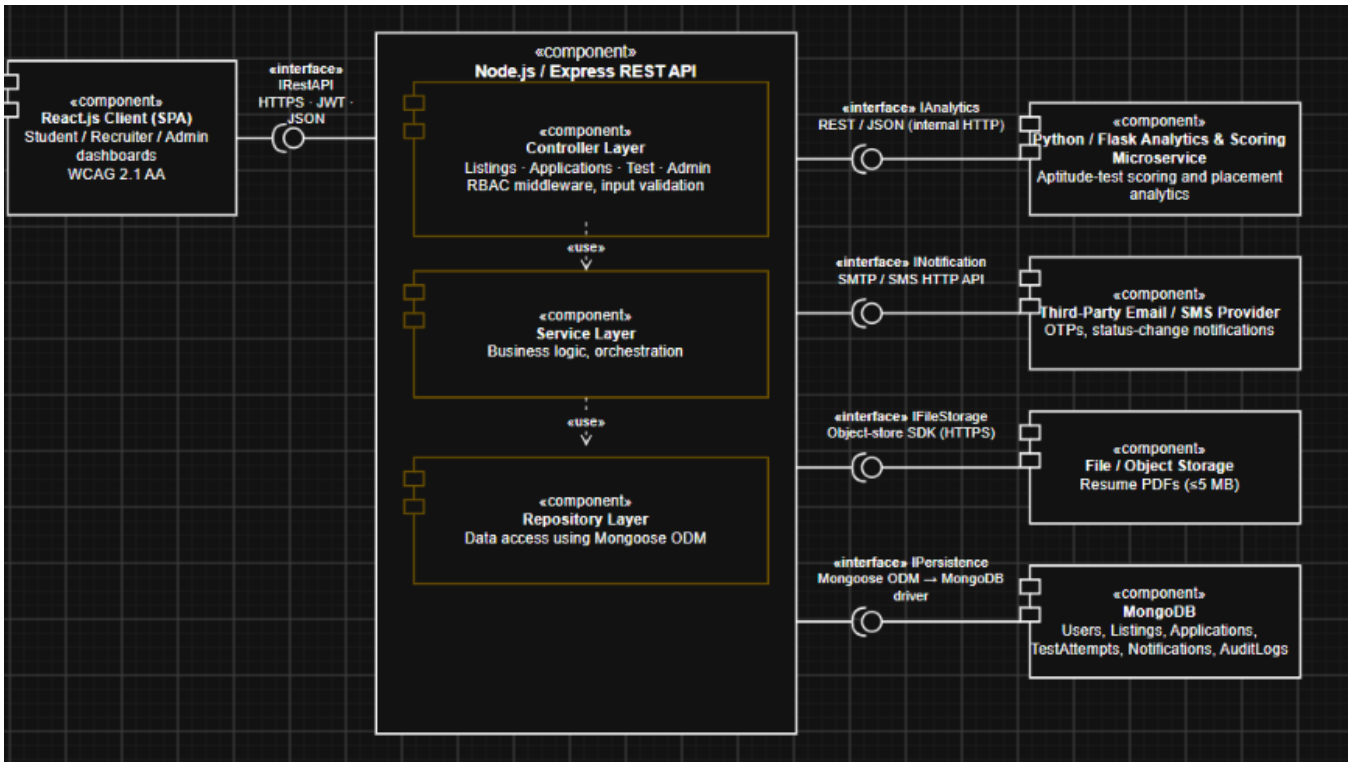


Figure 1: IPP Component Diagram

3.4 Component Descriptions

- React.js Client (SPA): role-based dashboards for Student, Recruiter, and Admin; consumes the REST API over HTTPS using JWT bearer tokens; implements WCAG 2.1 AA accessibility features.
- Node.js/Express REST API: the primary backend, organized into Auth, Listings, Applications, Test, and Admin controllers, with RBAC middleware enforced on every route; validates and sanitizes all inputs.
- Python/Flask Analytics & Scoring Microservice: evaluates aptitude-test submissions and computes test/placement analytics; invoked internally by the Express API over REST and isolated so it can be scaled or redeployed independently.
- MongoDB (via Mongoose ODM): persists Users, Listings, Applications, TestAttempts, Notifications, and AuditLogs collections.
- File/Object Storage: stores uploaded resume PDFs (≤5MB), referenced by Application records.
- Third-Party Email/SMS Provider: delivers registration OTPs and application status-change notifications.

3.5 Chosen Architecture Pattern and Rationale

IPP uses a layered (N-tier) architecture for the Node.js/Express API — Controller, Service, and Repository (Mongoose model) layers — combined with a single specialized Python/Flask microservice dedicated to aptitude-test scoring and analytics. The Express layers keep the bulk of CRUD-heavy, role-driven functionality (auth, listings, applications, admin) in one deployable unit, which is appropriate for an academic project of this scope and keeps deployment and debugging simple. The scoring/analytics logic is carved out into Python specifically because it benefits from Python's data-processing ecosystem and because isolating it limits the blast radius of failures and allows it to be scaled or iterated on independently of the main API. A full microservices decomposition (separate services per module) was considered and rejected as unnecessary operational overhead for this scale; a pure monolith was rejected because it would tie the data-analytics workload to the same runtime and deployment cadence as the core transactional API. The chosen hybrid — layered monolith plus one specialized microservice — was judged the best fit.

3.6 Technology Stack & Data Stores

- Frontend: React.js 18 (single-page application)
- Backend: Node.js 18+ / Express.js (REST API)
- Analytics/Scoring: Python 3.x / Flask microservice
- Primary data store: MongoDB Atlas (or self-hosted MongoDB) via Mongoose ODM — Users, Listings, Applications, TestAttempts, Notifications, AuditLogs collections
- Authentication: JWT (jsonwebtoken) with bcrypt/argon2 password hashing
- File storage: cloud/local object storage for resume PDFs
- Messaging: third-party email/SMS API (sandbox in test environments) for OTP and notifications
- Transport security: TLS 1.2+ on all client-server and server-to-microservice connections
- CI/CD: GitHub Actions; Testing: Jest/Mocha/Chai (JS), pytest (Python), Cypress/Selenium (E2E)

3.7 Risks & Mitigations

Risk	Mitigation
Python microservice downtime blocks test scoring	Queue/retry submissions; defer scoring and notify the student once the service recovers
MongoDB performance degradation under concurrent search load	Compound/text indexes on listing fields (IPP-F-007); paginate search results
Email/SMS provider outage delays OTP delivery	Retry-with-backoff strategy; sandbox/fallback provider for testing

Risk	Mitigation
Resume storage costs or limits exceeded	Enforce the 5MB cap at upload (IPP-SR-005); periodic archival of resumes for closed listings
RBAC misconfiguration exposes admin endpoints	Automated RBAC test suite (IPP-SR-006) run in CI; mandatory code-review checklist for new routes

3.8 Traceability to Requirements

Requirement ID(s)	Architectural Element
IPP-F-001 – IPP-F-005	Auth Controller (Express API) + MongoDB Users collection
IPP-F-006 – IPP-F-009	Listings Controller + MongoDB Listings collection
IPP-F-010 – IPP-F-013	Applications Controller + File Storage + Email/SMS Provider
IPP-F-014 – IPP-F-017	Test Controller (Express API) + Python Analytics & Scoring Microservice
IPP-F-018 – IPP-F-021	Dashboard read endpoints (Express API) + Python Analytics Microservice
IPP-F-022 – IPP-F-024	Admin Controller + RBAC Middleware + MongoDB AuditLogs collection
IPP-NF-001, IPP-NF-005	Express API layer design — stateless controllers enabling horizontal scaling
IPP-NF-003, IPP-SR-001 – IPP-SR-006	Security Architecture (Section 3.9)

3.9 Security Architecture

Threat Modeling (STRIDE):

STRIDE Category	Mitigation
Spoofing	Email/OTP-verified registration and JWT-based authentication for every session (IPP-F-001, IPP-F-002)
Tampering	Server-side input validation/sanitization on all endpoints; TLS 1.2+ in transit (IPP-SR-003)
Repudiation	Audit log of administrative actions with timestamp and actor ID (IPP-F-024)
Information Disclosure	TLS 1.2+ everywhere; password hashing (bcrypt/argon2); sensitive fields encrypted/hashed at rest (IPP-NF-003, IPP-SR-001)
Denial of Service	Rate limiting on authentication and file-upload endpoints; account lockout after 5 failed attempts (IPP-SR-004, IPP-F-004)
Elevation of Privilege	RBAC enforced at the API layer on every endpoint for Student/Recruiter/Admin roles (IPP-F-023, IPP-SR-006)

4. Design

4.1 Design Overview

IPP is designed as a layered Express API serving a React SPA, with a dedicated Python microservice for aptitude-test scoring and analytics. Each Express controller maps to a bounded area of functionality (auth, listings, applications, tests, admin) with its own Mongoose models, which keeps modules independently testable and traceable to SRS requirement IDs. The sections below walk through three representative end-to-end flows, the REST contracts for the busiest endpoints, and the cross-cutting concerns (error handling, logging, monitoring, UX, and open issues).

4.2 UML Sequence Diagrams

Three sequence diagrams are provided, covering authentication, the core application flow, and the aptitude-test/analytics flow that distinguishes IPP's architecture.

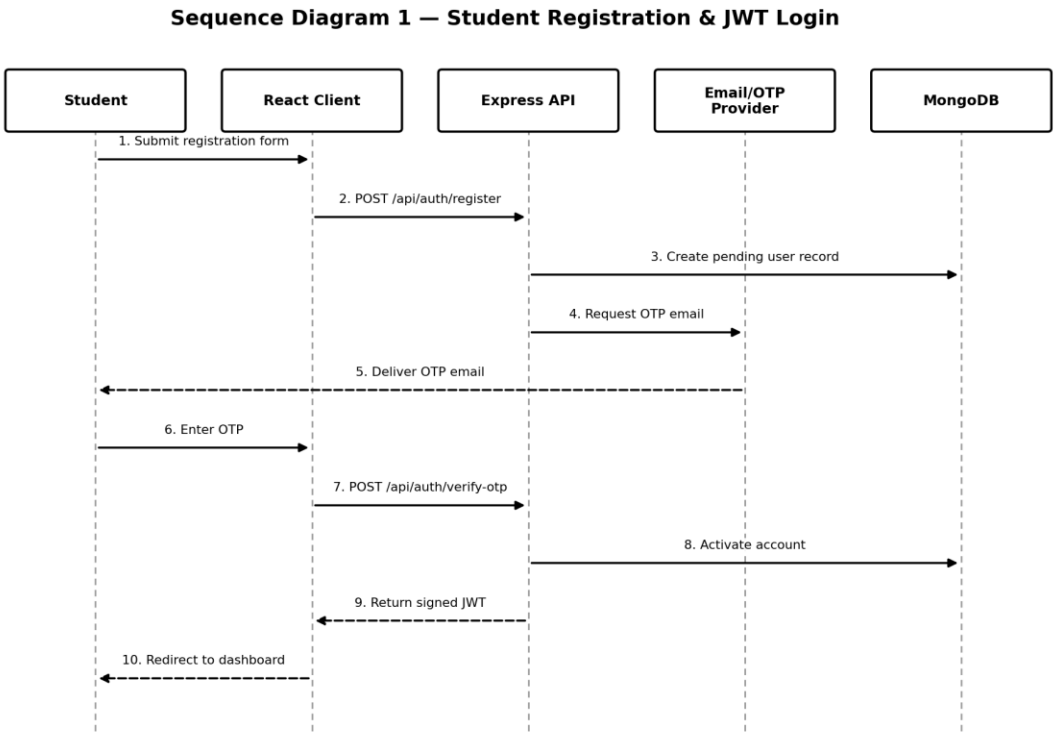


Figure 2: Sequence Diagram 1 — Student Registration & JWT Login

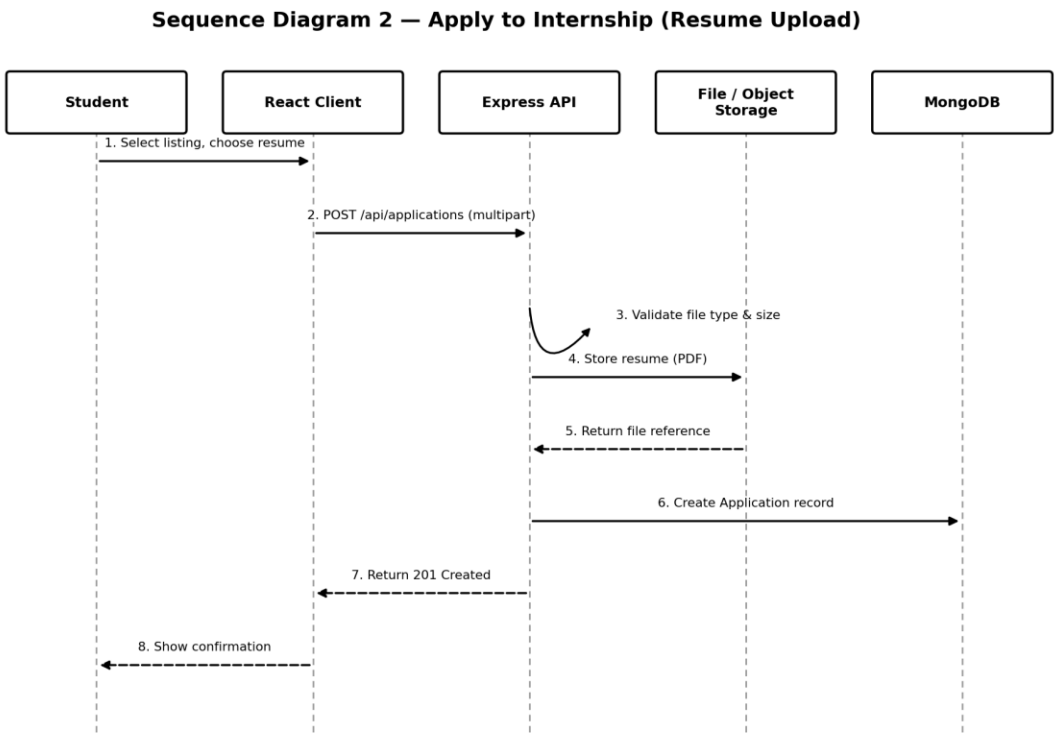


Figure 3: Sequence Diagram 2 — Apply to Internship (Resume Upload)

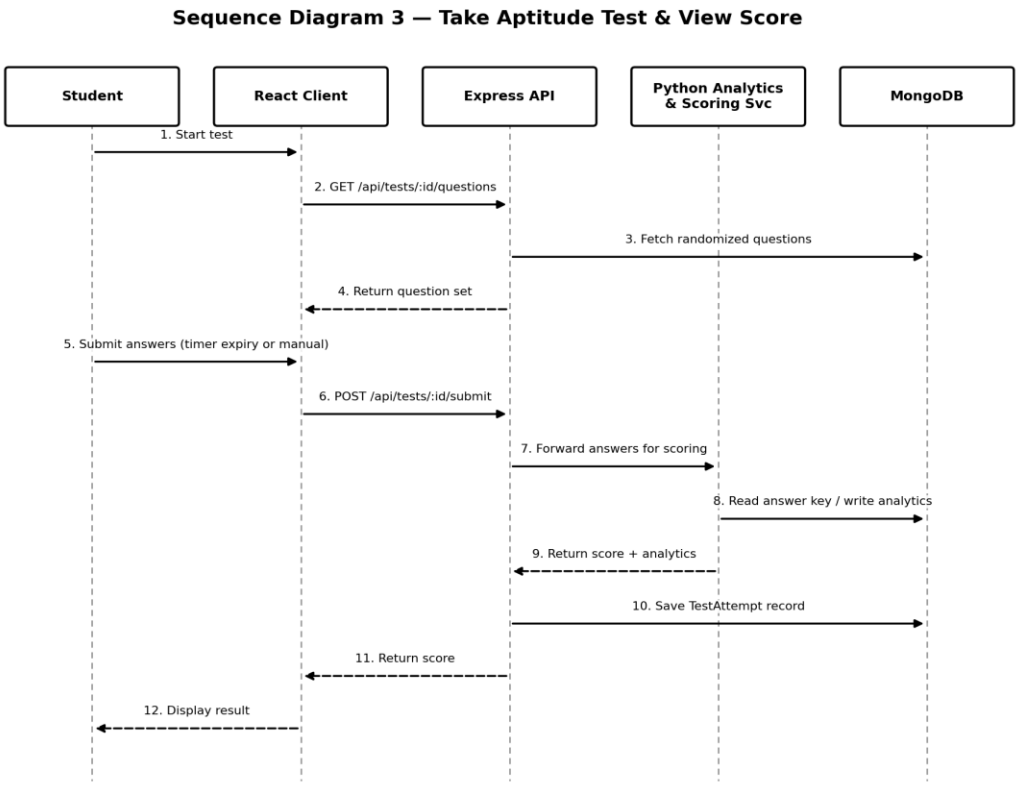


Figure 4: Sequence Diagram 3 — Take Aptitude Test & View Score

4.3 API Design

REST contracts for four representative endpoints spanning the Auth, Applications, and Analytics components.

POST /api/auth/register — Auth Controller

Field	Value
Method	POST
Request	{ name, email, password, role } — role ∈ {student, recruiter}
Response	201 { status: "pending_verification", userId }
Errors	400 Invalid input; 409 Email already registered

POST /api/auth/login — Auth Controller

Field	Value
Method	POST
Request	{ email, password }
Response	200 { token, expiresIn, role }
Errors	401 Invalid credentials; 423 Account locked (5 failed attempts)

POST /api/applications — Applications Controller

Field	Value
Method	POST (multipart/form-data)
Request	{ listingId, resume (PDF, ≤5MB) }
Response	201 { applicationId, status: "Applied" }
Errors	400 Invalid/oversized file; 409 Duplicate application; 410 Listing expired

POST /internal/analytics/score-test — Python Analytics & Scoring Microservice

Field	Value
Method	POST (internal, called by Express API only)
Request	{ attemptId, studentId, testId, answers[] }
Response	200 { score, accuracy, timePerQuestion[], categoryBreakdown }
Errors	400 Malformed payload; 422 Unknown question ID in answers

4.4 Error Handling, Logging & Monitoring

- All API error responses use a standardized JSON envelope: { status, code, message }; stack traces are never returned to the client.
- No sensitive information (passwords, JWTs, OTPs, resume contents) is written to logs; only request metadata, status codes, and sanitized error codes are logged.
- Centralized structured logging (e.g., Winston) on the Express API and the Python microservice, shipped to a log aggregator for correlation across services.
- Monitoring tracks: API response time (IPP-NF-001), monthly uptime (IPP-NF-002), failed-login/lockout counts (IPP-F-004), RBAC-violation counts (IPP-F-023), and Python microservice scoring latency.

4.5 UX Design

- Responsive React UI with distinct, role-based dashboards for Student, Recruiter, and Admin (IPP-F-018, IPP-F-019).
- WCAG 2.1 AA conformance: keyboard navigation, screen-reader-friendly labels, high-contrast mode (IPP-NF-004).
- Aptitude-test UI shows a persistent countdown timer and auto-saves progress periodically to tolerate intermittent connectivity.
- Mobile-first layout for browsing and filtering internship listings, since students are expected to check listings from mobile devices.

4.6 Open Issues & Next Steps

- Caching layer (e.g., Redis) for frequently filtered listing searches, to further support the IPP-NF-001 response-time target under higher load.
- Horizontal-scaling runbook for the Express API layer to formally validate IPP-NF-005 (2x load test).
- Future integration with third-party Applicant Tracking Systems (ATS), currently out of scope per the SRS.
- Future payment-processing support, should the platform introduce premium placement-prep features.

5. Appendices

5.1 Glossary

- IPP — Internship & Placement Preparation Portal
- MERN — MongoDB, Express.js, React.js, Node.js
- JWT — JSON Web Token
- RBAC — Role-Based Access Control
- OTP — One-Time Password
- API — Application Programming Interface
- TLS — Transport Layer Security
- WCAG — Web Content Accessibility Guidelines
- STRIDE — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
- ADR — Architecture Decision Record
- RTM — Requirements Traceability Matrix
- CSV — Comma-Separated Values

5.2 References

- IPP Software Requirements Specification (SRS) v1.0
- IPP Software Test Plan (STP) v1.0
- OWASP Top 10
- WCAG 2.1 AA Guidelines
- IEEE 42010 (Systems and Software Engineering — Architecture Description)

5.3 Tools

- Python (matplotlib) — component and sequence diagrams
- Postman — API design verification
- GitHub Actions — CI/CD pipeline
- OWASP ZAP — security scanning
- Lighthouse / axe — accessibility auditing
- Jira — sprint and task tracking

RESPONSIBILITIES & CONTRIBUTIONS

NAME	SRN	CONTRIBUTION
HARSHITH	PES2UG24CS186	Project understanding, scope, objectives and stakeholders
HEEMADHAWALA R	PES2UG24CS190	Technology, risks, requirements mapping and security

JOSHWIN PAUL	PES2UG24CS208	System architecture and UML component diagram
KANISHQ SURENDRAN	PES2UG24CS214	Detailed system design and API/interaction design